

Broker Research Entitlements PRD: With vs Without RequirementMentor

Audience: Product, engineering, compliance

Purpose: Show what changes when the same PM intent is drafted with org truth grounding vs a generic LLM draft

PM intent (identical for both):

Write a PRD for broker research entitlements. Add entitlement checks to generative search citations.

Capability area (org graph): Broker Research Entitlements

Primary touchpoints (org): Broker Research Entitlement Guard, Generative Answer Composer

Executive summary

	With RequirementMentor	Without RequirementMentor
What it is	A v1 product PRD that <i>extends</i> existing entitlement enforcement to generative search citations	A greenfield <i>entitlements platform</i> PRD (microservice, admin UI, APIs, infra)
Length	~2 pages, 12 focused FRs	~15 sections including API specs, DB schema, monitoring
Eng first reaction	Phasing, UX, audit shape — product debate	“We already have this — why are we rebuilding entitlements?”
Fact accuracy	Core §4 aligns with org truth; net-new is explicit	Most §4 reads as settled org fact but is invented
Validation	Passes accuracy gate (<code>handoff_ready=tr</code>	Would fail on invented requirements and

	ue) when drafted from kickoff themes	ungrounded metrics
--	--	--------------------

Bottom line: Same prompt, same PM skill — different outcome. RequirementMentor does not write the PRD for you. It constrains the first draft to what the org has already decided, and blocks invented “org requirements” before the PM shares v1.

What RequirementMentor provided (invisible to PM)

Before drafting, the PM saw a short kickoff — no claim IDs, no graph dumps:

Matched capability area: Broker Research Entitlements

What is already true for this area:

- Broker research access is governed by organization-wide entitlement rules
- Seat entitlements must be enforced before summarization and citations
- Generative search citations must respect broker research entitlements
- Compliance and audit requirements apply to how this capability is delivered
- Ingestion and query latency SLOs apply to this feature area

The agent then synthesized §4 from these themes. Validation ran silently against the truth packet before “ready to share.”

Side-by-side: scope and architecture

Dimension	With RM	Without RM
System model	Plug into existing Broker Research Entitlement Guard + Generative Answer Composer	New entitlements microservice between search and citation DB
Data store	Uses existing entitlement / subscription path	New PostgreSQL + Redis + 7-year audit store
Admin surface	Explicit non-goal — do not replace entitlement config workflows	FR4: full admin UI to grant/revoke, bulk

		import, self-service view
API design	Not specified (eng design phase)	Full REST appendix: POST /entitlements/check , grant/revoke/bulk-import
Failure mode	Flagged as “needs eng input”	“Safe mode: no citations returned” — major policy call, unstated as new
Rollout	Behind existing guard; phased (render → pre-summarization → GA)	17-week greenfield build across 5 phases

Side-by-side: §4 Functional Requirements

With RequirementMentor (excerpt — themes reflected)

FR	Requirement (summary)	Grounding
FR-1	Evaluate org-wide broker entitlements before broker content in generative results/citations	Org truth
FR-2	Resolve seat-level entitlements before summarization	Org truth
FR-3	Block/mask broker material when seat lacks entitlement — not after answer composed	Org truth

FR-4	Gate every sentence-level broker citation at render time	New (explicit in trust summary)
FR-5	Non-entitled citations not clickable/copyable; no broker metadata exposed	New
FR-6	Preserve sentence citations for entitled sources only	Org truth
FR-7	Auditable allow/deny: user, tenant, broker, timestamp, rule	Org truth
FR-8	Compliance admins review enforcement events without content access	New / needs eng input
FR-9	Org policy changes propagate via same path as direct broker access	Org truth
FR-10	Mixed entitled/non-entitled answers: no leak of titles/excerpts in body	New
FR-11	No material latency beyond existing feature-area budgets	Inherited SLO (qualitative)
FR-12	Hybrid retrieval + uniform gating	Context — may be scope creep

Without RequirementMentor (excerpt — what eng would challenge)

FR	Requirement (summary)	Problem
FR1	New entitlement data model; license expiration tracking	Invented — org already has entitlements
FR2	Query entitlements DB before every citation	Directionally OK; wrong integration point
FR3	Audit every access and filtered citation	Aligned in spirit; tied to new stack
FR4	Admin interface to grant/revoke; bulk import	Contradicts org scope — config is out of band
FR5	New API contract (POST /api/v1/entitlements/check , etc.)	Eng design in a PM doc; not grounded

Side-by-side: success metrics and NFRs

Metric / NFR	With RM	Without RM
Citation filtering	“Measurable per tenant” (qualitative)	“100% of citations filtered” — invented precision
Violations	Compliance outcome, not a fake zero	“Zero licensing violations” — ungrounded
Audit coverage	Auditable decisions per citation	“100% of events captured” — invented
Latency	Within existing feature-area SLO budgets	“<100ms”, “<50ms p95” — invented numbers

Availability	Not claimed	“99.9% uptime” for new service — invented
Scale	Not claimed	“10,000 checks/sec”, 50+ brokers — invented

RequirementMentor’s validation gate blocks ungrounded precise metrics (% , p95, ms) unless marked proposed or tied to graph claims.

What engineering would say on first review

With RequirementMentor

- “This extends the guard and composer — right initiative.”
- “FR-4/FR-5/FR-10 are new — let’s align phasing with rollout.”
- “FR-8 audit UI — is that v1 or v2?”
- “Latency: we have existing SLOs; let’s measure, not commit in PRD.”

Review type: Product judgment and phasing (~80% of comments).

Without RequirementMentor

- “We already have entitlements — this PRD scopes a duplicate system.”
- “Admin UI and bulk import are a different team/roadmap.”
- “Where did 50ms and 99.9% come from?”
- “Citation gating belongs in the composer/guard, not a new microservice.”
- “Safe mode = no citations — who approved that policy?”

Review type: Factual and scope correction (~60%+ of comments before product debate).

User story acceptance criteria

Criterion	With RM	Without RM
Every \$4 row supported by graph or marked new	Yes — trust summary separates grounded vs new	No — most rows presented as org fact

No §4 contradicts active org truth	Yes — extends existing guard	No — rebuilds entitlement stack
Inherited constraints in §4 or §6	Yes — guard, seat/org rules, SLOs, audit	Partial — audit yes; contradicts on admin/config
PM did not manually reconcile old PRDs / Slack / eng	Yes — kickoff from graph	N/A — would require eng to red-line
Eng back-and-forth on facts vs product choices	Low on facts	High on facts

What each doc is good for

Doc	Best use	Do not use for
With RM	Share with eng/compliance as v1 intent; phase and refine	API design, schema design (defer to tech design)
Without RM	Illustrate generic LLM failure mode; training example	Shipping, estimation, architecture review

Recommendation for the team

1. **Treat the RM draft as the v1 product artifact** — extend existing systems, explicit non-goals, qualitative metrics.
 2. **Use the no-RM draft as the control arm** — same intent shows why a truth layer matters.
 3. **Before external demo:** ensure kickoff truth summary has no graph jargon; validate silently before “ready to share.”
 4. **Dogfood metric:** % of §4 FRs eng accepts as factually aligned on first review (target ≥80% on broker).
-

Appendix: trust summary (With RM)

Grounded in org context: Organization-wide entitlement rules, seat enforcement before summarization/citations, generative citation gating, compliance audit requirements, hybrid retrieval SLOs.

New in this PRD: Citation-render-time gating, mixed-source leak prevention in answer bodies, compliance-admin review of citation enforcement without content access.

Needs eng input: Entitlement-denied UI states, caching strategy for latency SLOs, audit log schema/retention.

Document generated for internal team alignment. Feature: Broker Research Entitlements + generative search citation checks. RequirementMentor = truth layer between requirement graph and LLM.